CogLean in five slides

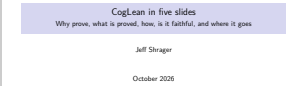
Why prove, what is proved, how, is it faithful, and where it goes

Jeff Shrager

October 2026

CogLean in five slides

2026-10-28



- One straight-through talk: the core slide from each of the five CogLean decks. Each slide's notes begin with the transition, then the original notes.

Three kinds of evidence a theory can give

One run	**Every run**	**Every model satisfying the postulates**
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

Three kinds of evidence a theory can give

One run	Every run	Every model satisfying the postulates
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

Transition. Why prove things about cognitive theories at all.

- A theorem quantified over every model satisfying the postulates says which assumptions matter (the proof uses some and not others), what is robust, and whether a claim is possibility or necessity.
- Simulation is still essential: it explores, suggests conjectures, and fits data. Proof settles which conjectures follow from which assumptions. Experiments with people decide whether the assumptions are right.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^n$.

In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^n$.

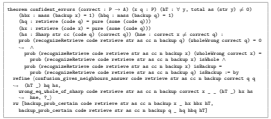
In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Transition. What gets proved: a prediction that neither EPAM nor the distributions-of-associations model makes alone, holding for every parameter setting. (Full talk: recognition-retrieval.)

- This is the prediction neither theory makes alone.
- Two cautions. The model has no timing, so "fast" would need a timing model. And the errors are frequent only if retrieval on q usually succeeds (R_q near 1); if $R_q = 0$, every trial uses the backup.

Confident errors: the proof is a composition

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    prob (recognizeRetrieve code retrieve str as cc n backup q) (wholeWrong correct q) = 0
    ↪ ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) (wholeWrong correct x) =
        prob (recognizeRetrieve code retrieve str as cc n backup x) isWhole ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) isBackup =
        prob (recognizeRetrieve code retrieve str as cc n backup q) isBackup := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup correct q q
  ↪ (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup correct x _ _ (hT _) hx hs
    ↪ hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]
```



```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total_as (otr y) ≠ 0)
    (hbin : mass (backup x) = 1) (hdeg : mass (backup q) = 1)
    (hq : retrieves (code q) = pure (some (code q)))
    (hx : retrieves (code x) = pure (some (code q)))
    (hne : ¬∃ansy otr ∈ (code q).Correct q)) (hne : correct x ≠ correct q) :
  prob (recognizeRetrieve code retrieve str as cc x backup q) (fallsAlong correct q) = 0 :=
by
  refine (confusion_gives_neighbours_answer code retrieve str as cc x backup correct q q
    (hT _) hq hx
    (wrong_eq_choice_of_sharp code retrieve str as cc x backup correct x _ (hT _) hx hne
      (hne, _))
    (backup_prob_certain code retrieve str as cc x backup x _ hx hbin hT)
    backup_prob_certain code retrieve str as cc x backup q _ hq hdeg hT)
```

Transition. How it is proved: the actual Lean proof is five lines, because the work is factored into general lemmas below it. (Full talk: recognition-retrieval-lean.)

- Three conjuncts, three earlier lemmas. The first is the neighbour's-answer lemma applied to q itself; the second says that on sharp associations, every stated answer to x is wrong; the third rewrites both backup probabilities to the same closed form $(1 - R_q)^n$, after which the goal is an identity.
- Note the hypotheses: retrieval is certain (`= pure (some ...)`). That is exactly what ACT-R never satisfies, and what the laws later supply.

Results: no failures

Check	Lean theorem	kind	tests	failures
Lean exact	`#eval MD.trial`	prob	243	0
P1	`columns_prob`	prob	394	0
P2	`whole_depends_only_on_route`	prob	8543	0
P3	`confusion_gives_neighbours_answer`	exact	200	0
P4	`confident_errors`	exact+prob	222	0
P7	`learning_preserves_trial`	exact	74692	0
P9	`columns_place_value`	exact	248800	0
P11	`discrimination_removes_confusion`	exact	91090	0
P6a, P6b, P14, P17, P18			0	*vacuous*

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

Results: no failures

Check	Lean theorem	kind	tests	failures
Lean exact	#eval RD.trial	prob	243	0
P1	columns_pcrh	prob	394	0
P2	whole_depends_only_on_route	prob	8543	0
P3	confusion_gives_neighbours_answer	exact	200	0
P4	confident_errors	exact+prob	222	0
P7	learning_preserves_trial	exact	74602	0
P9	columns_place_value	exact	248800	0
P10	discrimination_removes_confusion	exact	91090	0
P8a, P8b, P14, P17, P18			0	vacuous

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

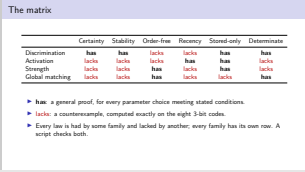
Transition. Is the formalization faithful? A Python simulator written blind from a prose specification, against predictions preregistered before it existed. (Full talk: blind-simulation.)

- A selection of rows; the full table (22 checks) is in `MultiDigit/sim/RESULTS.md`.
- "Lean exact": 243 outcome frequencies on the four shared instances, each within tolerance of Lean's exact probability.

The matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

- **has**: a general proof, for every parameter choice meeting stated conditions.
- lacks: a counterexample, computed exactly on the eight 3-bit codes.
- Every law is had by some family and lacked by another; every family has its own row. A script checks both.



Transition. Where it goes: instead of formalizing models one by one, laws of memory, and a matrix of which families have which law, every cell a theorem. Experiments can then be derived from laws that disagree. (Full talk: laws-matrix-experiments.)

- Examples of "has": EPAM's certainty needs only that the items be noticeably different; ACT-R's recency holds for any positive mismatch discount and failure weight (the later item is retrieved exactly twice as often); strength memory and MINERVA are order-free for every parameter choice.
- A "lacks" cell says only that the law fails somewhere. Mapping the parameter regions where a family does have it is open.